1. Complexity and Big-O Notation

Big-O notation describes how the running time or memory use of an algorithm grows as the input size n grows, ignoring constant factors and lower-order terms. It gives an upper bound on the growth rate, usually for the worst case.

Common classes from fastest to slowest are O(1) constant, O(log n) logarithmic, O(n) linear, O(n log n), O(n^2) quadratic and O(2^n) exponential. Binary search on a sorted array is O(log n) because it halves the search range at every step.

Time complexity counts operations, while space complexity counts extra memory. There is often a trade-off: storing precomputed results uses more memory but saves time.

Amortised analysis looks at the average cost per operation over a long sequence of operations, which is useful when occasional operations are expensive but most are cheap.

2. Arrays and Linked Lists

An array stores elements in one contiguous block of memory, so any element can be read by index in O(1) time. Inserting or deleting in the middle is O(n) because later elements must be shifted.

A dynamic array, such as a Python list, grows automatically. When it is full it allocates a larger block, typically double the size, and copies the elements across. A single resize costs O(n), but appending is O(1) amortised because resizes are rare.

A singly linked list stores each element in a node that holds a pointer to the next node. Inserting or deleting at a known node is O(1), but accessing the i-th element requires walking the list from the head, which is O(n). A doubly linked list also stores a pointer to the previous node, which makes deletion and backward traversal easy.

Linked lists have poor cache locality because nodes are scattered in memory, so arrays are often faster in practice even when the Big-O costs look similar.

3. Stacks and Queues

A stack is a last-in, first-out (LIFO) structure: the last element pushed is the first one popped. Push, pop and peek are all O(1). The call stack that stores function calls and local variables is a stack, and stacks are also used to check balanced brackets and to evaluate expressions.

A queue is a first-in, first-out (FIFO) structure: elements are added at the back (enqueue) and removed from the front (dequeue). Queues model waiting lines and are used by breadth-first search.

Implementing a queue with a plain array makes dequeue O(n) because every element shifts forward. A circular buffer avoids this by keeping front and back indices that wrap around the end of a fixed-size array, giving O(1) enqueue and dequeue.

A deque (double-ended queue) supports adding and removing at both ends in O(1).

4. Hash Tables

A hash table stores key-value pairs and supports insert, lookup and delete in O(1) time on average. A hash function converts each key into an index in an underlying array of buckets.

Two different keys can map to the same index; this is called a collision. With separate chaining, each bucket holds a list of all entries that hash to it. With open addressing, a colliding entry is placed in another free slot found by probing, for example linear probing checks the next slots one by one.

The load factor is the number of stored entries divided by the number of buckets. As it rises, collisions become more frequent and operations slow down, so the table is resized, typically doubled, and all entries are rehashed when the load factor passes a threshold such as 0.75.

In the worst case, when many keys collide, operations degrade to O(n). A good hash function spreads keys uniformly across the buckets.

5. Binary Search Trees and Balancing

In a binary search tree (BST), every node's left subtree contains only smaller keys and its right subtree only larger keys. This ordering lets search, insert and delete follow a single path from the root, costing O(h) where h is the height of the tree.

If keys are inserted in sorted order, the tree degenerates into a chain that behaves like a linked list, its height becomes n, and operations become O(n). An in-order traversal of a BST visits the keys in sorted order.

Self-balancing trees keep the height at O(log n). An AVL tree stores a balance factor for each node, the difference between the heights of its subtrees, and restores balance with rotations whenever the factor leaves the range -1 to 1.

Red-black trees use node colours and a few rules to stay approximately balanced. They do fewer rotations than AVL trees on insertion and are used inside many standard library map implementations.

6. Heaps and Priority Queues

A binary heap is a complete binary tree that satisfies the heap property: in a min-heap every parent is smaller than or equal to its children, so the minimum is always at the root. In a max-heap the largest element is at the root.

Because the tree is complete, it is stored compactly in an array with no pointers. For the node at index i, the left child is at index 2i+1, the right child at 2i+2, and the parent at index (i-1)//2.

Insertion appends the new element at the end and sifts it up while it is smaller than its parent. Removing the root moves the last element to the top and sifts it down. Both cost O(log n), while reading the minimum is O(1). Building a heap from an unsorted array with heapify takes only O(n).

A heap is the natural implementation of a priority queue. To find the k largest of n items, keep a min-heap of size k: each new item that beats the root replaces it, so the whole selection costs O(n log k) instead of O(n log n) for a full sort.

7. Graphs and Shortest Paths

A graph consists of vertices connected by edges, which can be directed or undirected and weighted or unweighted. An adjacency matrix uses O(V^2) memory and answers 'is there an edge between u and v?' in O(1). An adjacency list uses O(V + E) memory and is preferred for sparse graphs.

Breadth-first search (BFS) explores the graph level by level using a queue, and finds shortest paths in an unweighted graph. Depth-first search (DFS) goes as deep as possible along each branch using a stack or recursion, and is used for cycle detection and topological sorting.

Dijkstra's algorithm finds the shortest paths from one source vertex in a graph with non-negative edge weights. It repeatedly takes the unvisited vertex with the smallest known distance from a priority queue and relaxes its outgoing edges. With a binary heap it runs in O((V + E) log V).

Dijkstra's algorithm does not work with negative edge weights; the Bellman-Ford algorithm handles them and can also detect negative cycles.

8. Sorting Algorithms

Merge sort divides the array into halves, sorts each half recursively and merges the two sorted halves. It always runs in O(n log n) time, is stable, which means equal elements keep their original order, but needs O(n) extra memory for merging.

Quicksort picks a pivot, partitions the array into elements smaller and larger than the pivot, and sorts the two parts recursively. It averages O(n log n) and sorts in place, but its worst case is O(n^2) when the pivot is repeatedly the smallest or largest element, for example on already sorted input with a first-element pivot. Choosing a random pivot or the median of three makes the worst case very unlikely.

Comparison-based sorting cannot beat O(n log n) in the worst case. Counting sort avoids comparisons and runs in O(n + k) for integer keys in a small range of size k.

Insertion sort is O(n^2) in general but very fast on small or nearly sorted arrays, so practical sorting routines often switch to it for small subarrays.